GATE
CS & DA

Artificial Intelligence

Un-informed Search

Lecture No. 5

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

Topic DFS

Topic DLS

Topic IDDFS

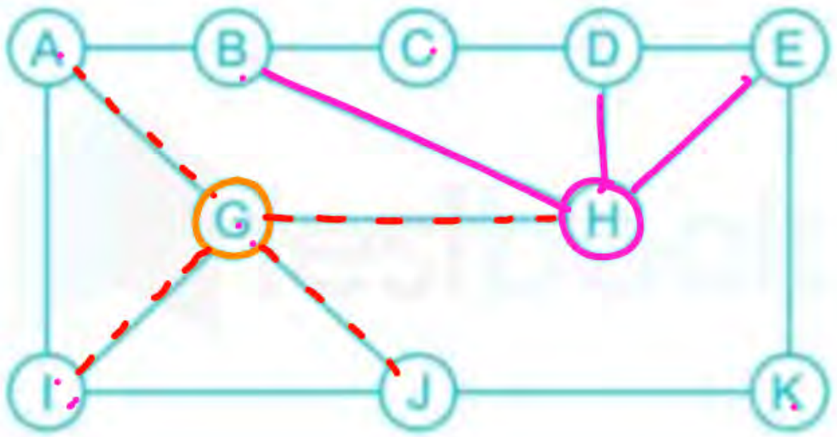
Topic UCS

Topic

Topics to be Covered

- Topic — UCS
- Topic — Question (BFS/DFS/UCS)
- Topic
- Topic
- Topic

Consider the following graph.



BFS

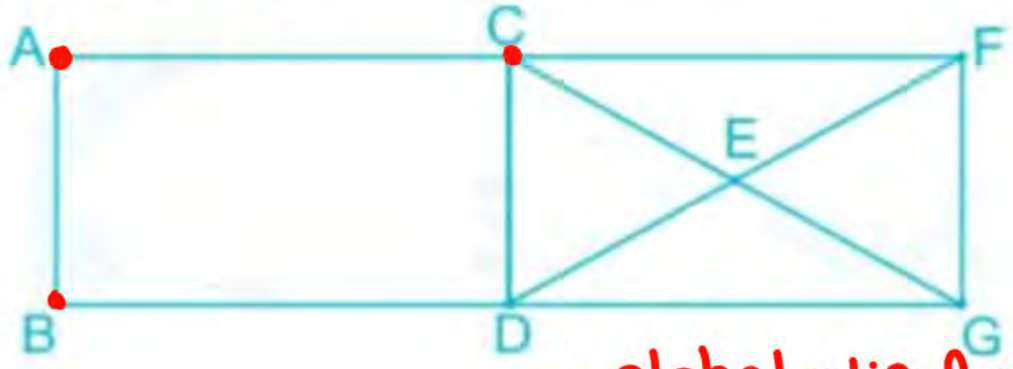
GAHIJBDEKC ← F1

GAHIJBDEKC

Apply breadth first search traversal of the above graph. Which of the following traversal is possible if start vertex is G? (Assume lexicographic ordering) ← alphabetical

1. GAHJIBCEKD
2. GAHJIBCDEK
3. GAHIJBDEKC ✓
4. GAHIJBCEKD

Which sequence cannot be obtained while using BFS traversal for the below given graph?



• alphabetical order (not mention)

I. A B C F G E D → ABC

II. A B C G F E D → ACB

III. A B D E C F G (III) not BFS.

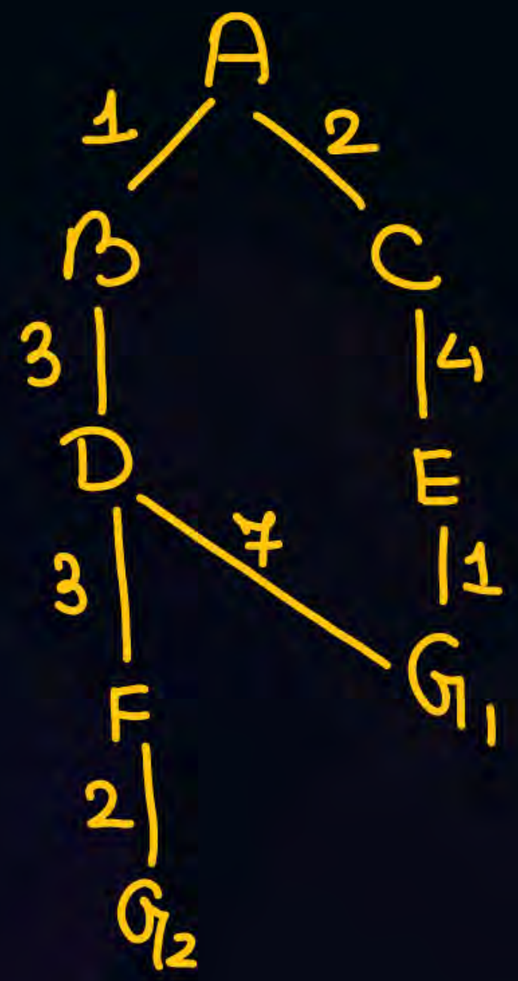
Compare BFS and DFS

UCS

	A	(A,B)	(A,B,C)	(A,B,C,D)		
A	0	0	0	0	0	0
B	1	1	1	1	1	1
C	2	2	2	2	2	2
D	∞	4	4	4	4	4
E	∞	∞	6	6	6	6
F	∞	∞	∞	7	7	7
G_1	∞	∞	∞	11	7	7
G_2	∞	∞	∞	∞	∞	9

D E

Compare BFS and DFS



BFS → [alphabetical]

DFS →

UCS →

- BFS: ~~A~~ ~~B~~ ~~C~~ ~~D~~ ~~E~~ ~~F~~ G1 G2

  G1 ← goal node.

- DFS: ~~A~~ C ~~B~~ ~~D~~ G1 ~~F~~ G2 | goal node.

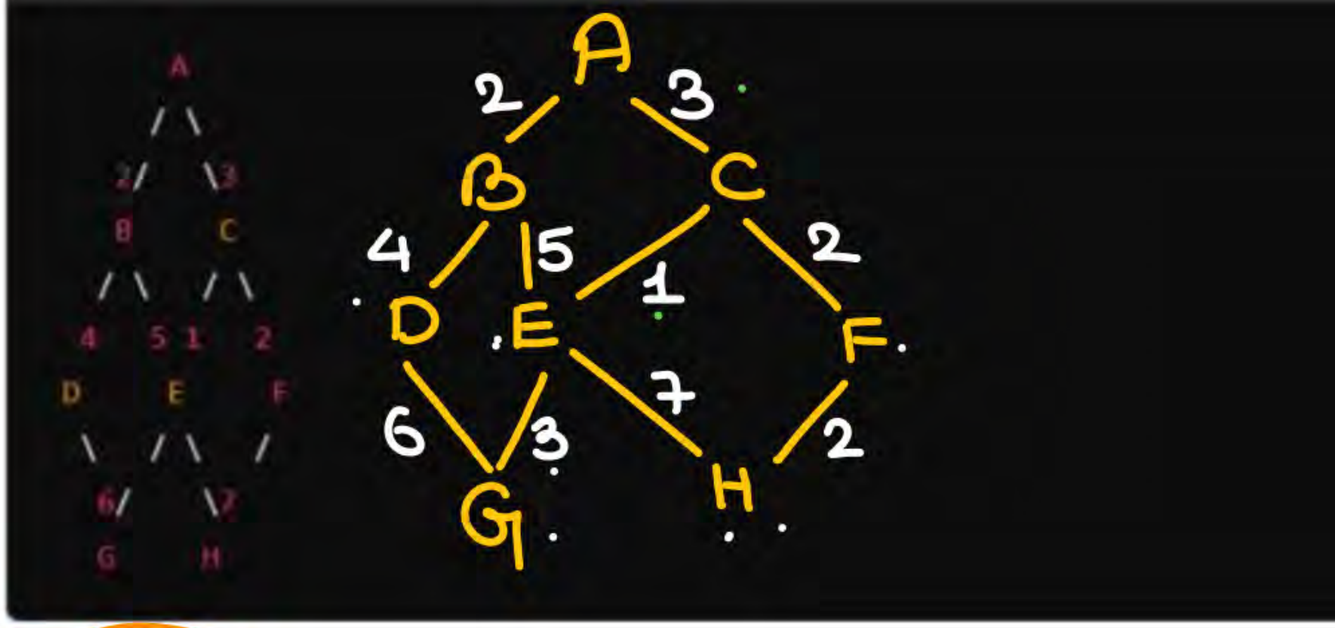
```
          A
         / \
       1/   \3
       B     C
      / \   / \
    4   2 5   6
   /     X     \
  D----3/ \1-----G
 / \    |   |   / \
5   3  (G1) 2  4   7
|   |    |   | /   |
E--F----I---J    (G2)
 \   \  / \  /
  6   2/   3
    \  |   /
     \ | /
    (G3) H
```

	A	B	C	E	F
A	0	0	0	0	0
B	2	2	2	2	2
C	3	3	3	3	3
D	∞	6	6	6	6
E	∞	7	4	4	4
F	∞	∞	5	5	5
G	∞	∞	∞	7	7
H	∞	∞	∞	11	7

A ← C ← F ← H

Question: Consider the following graph:

mathematica



Goal Node: `H`

Question: What is the cost of the path found by UCS from node `A` to node `H`?

A) 7

B) 10

C) 11

D) 14

	A	B	D	C
A	0	0	0	0
B	1	1	1	1
C	4	4	4	4
D	0	3	3 → 3	
G	0	8	8 → 8	

A → B → G

A → G: 8

A → B → D

A → C

B → G

Question: Consider a graph where the edge costs are as follows:

- A -> B: 1
- A -> C: 4
- B -> D: 2
- C -> D: 1
- D -> G: 5
- B -> G: 7
- C -> G: 6

What is the first goal node found by UCS starting from node `A`?

A) G

B) D

C) C

D) B

Uniform Cost search

Uniform Cost Search (UCS) is a search algorithm used to find the least-cost path from a start node to a goal node in a graph. UCS is a variant of Dijkstra's algorithm and is used in situations where the path costs vary, making it more suitable for weighted graphs.

DFS
BFS → non weighted graph

Uniform Cost search

Key Concepts

Priority is given to node with least distance

- Priority Queue: UCS uses a priority queue to explore the least-cost path first. Nodes are expanded in order of their cumulative cost from the start node.
- Cost: Each edge in the graph has a cost associated with it. UCS aims to find the path with the minimum cumulative cost.
- Expansion: The algorithm expands the node with the smallest cumulative cost, ensuring that the cheapest path is always chosen.

- H, F, I are goal

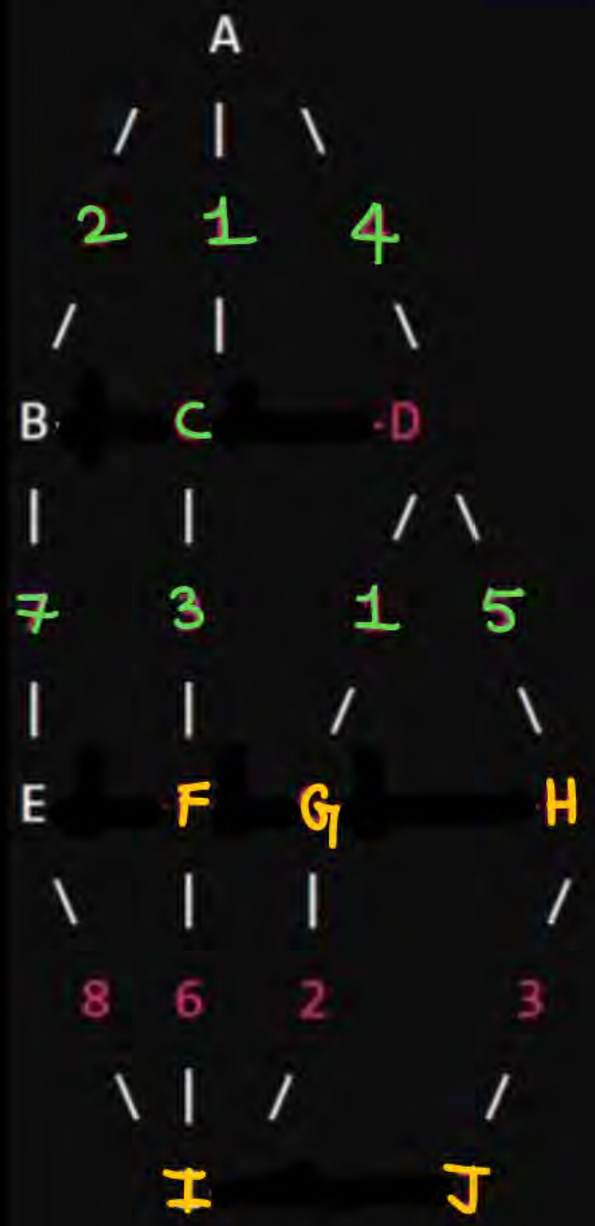
DFS ⇒ ~~A~~ D C ~~B~~ ~~E~~ I

BFS ⇒ A ~~B~~ ~~C~~ ~~D~~ ~~E~~ F G H I

IDDFS ⇒ It 1 A

It 2 A ~~B~~ ~~C~~ ~~D~~

It 3 A D ~~B~~ ~~C~~ ~~E~~ ~~F~~



```
        A
      / | \
    2   1   4
   /    |    \
  B----C------D
  |    |      / \
  7    3     1   5
  |    |    /     \
  E----F---G-------H
   \   |   |      /
    8  6   2     3
     \ | /     /
       I-------J
          9
```

- H, F, I are goal

	A	C	B	D	F	G
A	0	0	0	0	0	0
B	2	2	2	2	2	2
C	1	1	1	1	1	1
D	4	4	4	4	4	4
E	∞	∞	9	9	9	9
F	∞	4	4	4	4	4
G	∞	∞	∞	5	5	5
H	∞	∞	∞	9	9	9
I	∞	∞	∞	∞	10	7
J	∞	∞	∞	∞	∞	∞

A→C→F

4

- H, F, I are goal

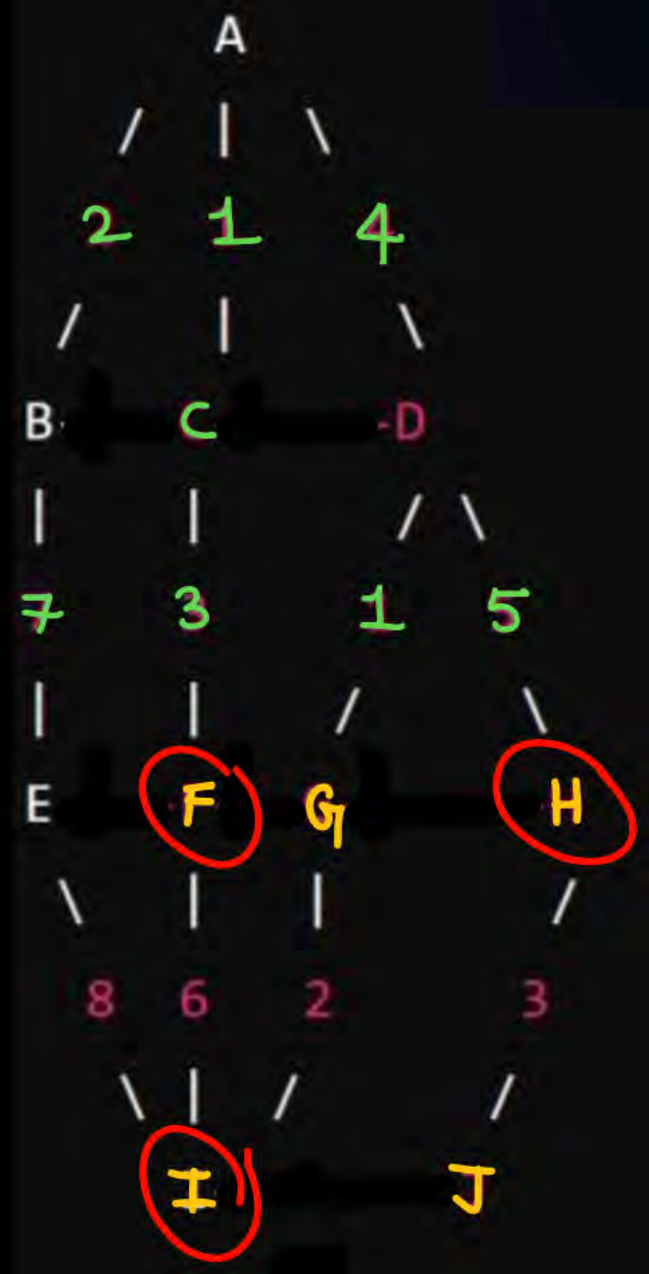
alpha betical order.

DFS ___

BFS ___

IDDFS ___

UCS ___



1. Time Complexity

- **Nodes Expanded**: UCS explores all possible paths by expanding nodes based on their cumulative cost from the start node. In the worst case, it might need to explore nearly every node in the search space.

worst Case ⇒ depth ⇒ $\frac{\text{optimal Cost}}{\text{min weight}}$

- **Upper Bound**: The time complexity is $O(b^{C^*/\epsilon})$, where:
  - b is the branching factor (maximum number of children per node).
  - C^* is the cost of the optimal solution.
  - ϵ is the smallest edge cost in the graph.

$T.C \Rightarrow O(b^{C^*/\varepsilon})$

- **Explanation**: The algorithm expands nodes in order of increasing path cost. If the path to the goal is long, UCS may need to explore many nodes before it finds the optimal path. The factor C^*/ϵ represents the number of times the smallest edge weight must be traversed to reach the goal, which can be a large number.

2. Space Complexity

UCS ⇒ optimal

- **Memory Usage**: UCS needs to keep track of all nodes in the frontier (open list) to explore the least costly one next. It also needs to store the entire path for each node.
- **Upper Bound**: The space complexity is also $O(b^{C^*/\epsilon})$, as the frontier can grow exponentially with the depth of the search, similar to the time complexity.
- **Explanation**: The space complexity is dominated by the size of the frontier, which can become large if the path to the goal is deep and involves many low-cost nodes.

Note: UCS can be computationally expensive in terms of both time and space, especially when dealing with large graphs or when the cost difference between paths is minimal (i.e., when ϵ is small). However, UCS is guaranteed to find the optimal solution if it exists and all costs are non-negative.

Uniform Cost search

- Algorithm Steps
- Initialization: Initialize the priority queue with the start node, assigning it a path cost of 0. Set the cost of reaching the start node to 0.
- Expand Nodes:
- While the priority queue is not empty, remove the node with the lowest path cost from the queue.
- If the removed node is the goal node, return the path and the total cost.
- For the removed node, consider all its neighbors and calculate their path costs.
- If a neighbour has not been explored or a cheaper path to it is found, update its cost and add it to the priority queue.

Uniform Cost search

not Imp

- Applications
- Pathfinding in artificial intelligence and robotics.
- Network routing protocols.
- Cost optimization problems in logistics and transportation.

Uniform Cost search

Limitations

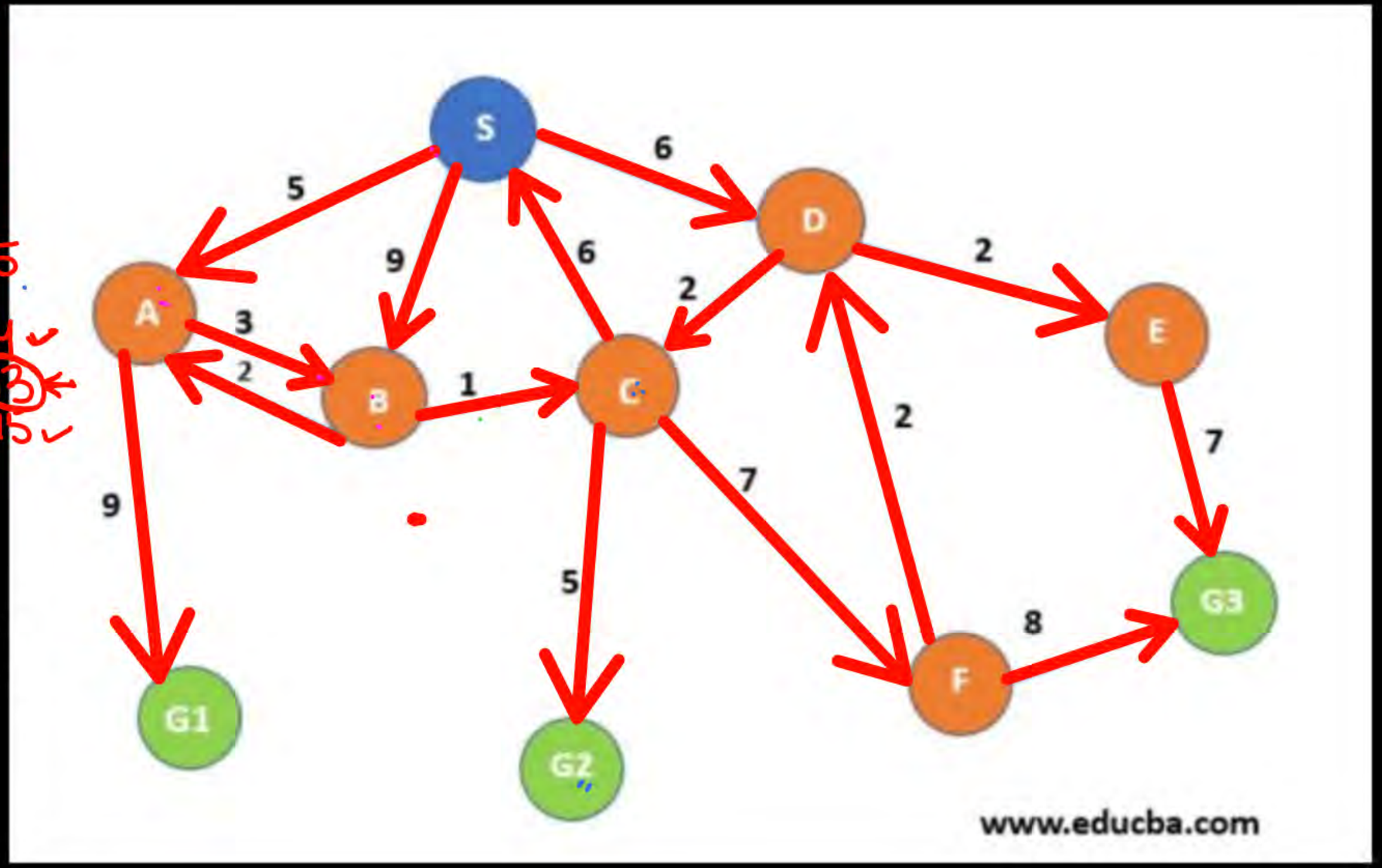
bcoz it explore all path.

- UCS can be less efficient with large graphs due to its memory and time requirements.
- The algorithm assumes non-negative edge costs; negative costs can lead to infinite loops or incorrect results.

	S	A	D	B	C	E
S	0	0	0	0	0	0
A	5	5	5	5	5	5
B	9 →	8	8	8	8	8
C	∞	∞	8	8	8	8
D	6	6	6	6	6	6
E	∞	∞	8	8	8	8
F	∞	∞	∞	∞	15	15
G_1	∞	14	14	14	14	14 ✓
→ G_2	∞	∞	∞	∞	13	13 ←
G_3	∞	∞	∞	∞	∞	15 ✓

G_2 is Ans 13

$S \leftarrow D \leftarrow C \leftarrow G_2$



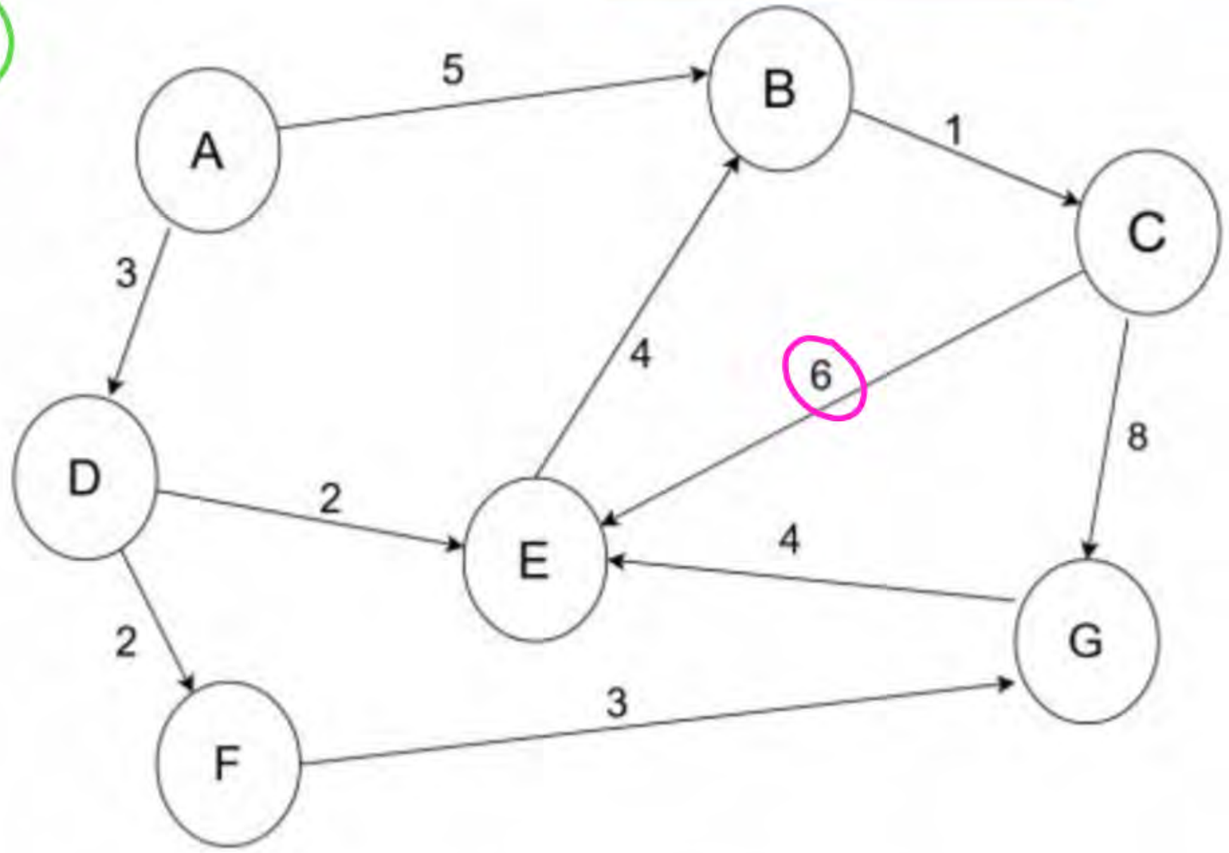
Uniform Cost search

	A	D	F	E	B	C
A	0	0	0	0	0	0
B	5	5	5	5	5	5
C	∞	∞	∞	∞	6	6
D	3	3	3	3	3	3
E	∞	5	5	5	5	5
F	∞	5	5	5	5	5
G	∞	∞	8	8	8	8

A ← D ← F ← G

Consider the following graph. Let the starting node be A and the goal node be G.

8



Finding path A to G using uniform cost search

Uniformed Search

THANK - YOU